TripMate AI: Intelligent Group Activity & Travel Planning Platform

Unified AI Agent Architecture, Cloud Deployment & Production Engineering

Technical Project Proposal for Academic Supervisor | Group Project Concept

Project Team (6 Members): Ta Quang Huy Ha Dang Huy Pham Dinh Anh Duong Nguyen Minh Duc Nguyen Tung Duong Dinh Tien Luan

1. Introduction, Scope & Product Overview

1.1. The Problem: Pain Points in Group Trip Planning

Coordinating leisure outings or vacations with friends is notoriously friction-heavy:

- **Tool Fragmentation:** Planning requires juggling 5+ disconnected applications (Google Maps, review blogs, Agoda, Booking.com, chat apps).
- **Preference Deadlocks:** Conflicting budgets, dietary restrictions, and activity styles stall decision-making.
- **Organizer Burnout & Buried Context:** Chat polls and venue links quickly get buried under hundreds of messages, leaving a single person overwhelmed with building and adjusting schedules.

1.2. Target Users & Practical Scope

- **Target Users:** Small groups (2–10 friends, university students, coworkers) organizing weekend city hangouts or multi-day vacations.
- **Practical Scope:** Domestic travel in Vietnam and visa-free ASEAN destinations, focusing strictly on intelligent scheduling, venue optimization, and real-time collaboration without visa complexities.

1.3. The Solution: Two-Stage Consensus Planning Flow

TripMate AI unifies group planning through a collaborative room powered by a **single, unified AI Agent (DeepSeek-V4)** that guides the group through two consensus checkpoints (Figure 1):

1. **Desires Capture:** Members share dates, budgets, and vibes in persistent room chat, saved directly into PostgreSQL.
2. **Macro Consensus:** The AI drafts high-level dates and daily themes; the group discusses and votes to lock the roadmap.
3. **Micro Curation & Stop Voting:** The AI queries live APIs (Google Places, Agoda, Klook) to populate venues clustered within a 2–3 km radius per half-day block. Members vote thumbs UP/DOWN, with 1-click alternative swaps.
4. **Final Confirmation:** The Host confirms the finalized plan, generating PDF and calendar (.ics) exports with synchronized Mapbox routes.

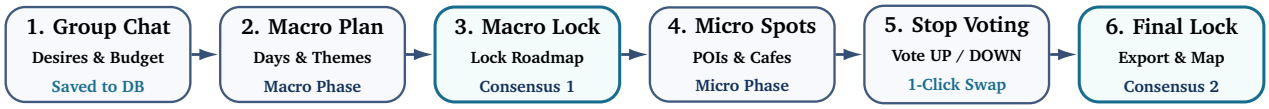

Figure 1: Two-stage consensus planning flow: gather desires → macro roadmap → lock macro → curate micro spots → granular vote → final plan lock

2. Platform Features & User Experience

2.1. Dual Planning Scopes

- **Day Hangouts:** Rapid curation for spontaneous outings (e.g., *"cafe → board game lounge → hotpot"*), prioritizing currently open venues with minimal transit times.
- **Multi-Day Vacations:** Complete travel itineraries (e.g., *"4 days in Da Nang & Hoi An"*) combining hotel recommendations, daily meal spots, sights, and optimized routes.

2.2. Shared Collaborative Room & Live Map Canvas

- **Invite via Link & Role Access:** Instant room entry with *Host* (plan locking, settings) and *Member* (chat, stop voting, swaps) permissions.

- **Interactive Timeline & Mapbox GL:** Draggable stop cards displaying photos, opening hours, estimated stay duration, and live Mapbox route navigation.

2.3. Two-Tier Consensus Engine

- **Stage 1 (Macro Approval):** Group locks the high-level roadmap before any expensive venue API calls are triggered.
- **Stage 2 (Micro Stop Voting):** Individual stops are voted UP/DOWN. Downvoted venues immediately present 2 pre-fetched backup options with 1-click swaps.
- **Living Itinerary:** Final plan confirmation persists normalized records to PostgreSQL but leaves the itinerary dynamic for real-time on-trip adjustments.

3. The AI Agent: Architecture, Workflow & Grounding Pipeline

3.1. Unified AI Agent: DeepSeek-V4 with Two-Phase State Execution

TripMate AI utilizes a **single, unified AI Agent powered by DeepSeek-V4** in LangGraph. To resolve latency bottlenecks and ensure clean testability, the agent operates in two decoupled state phases:

- **Phase 1 (Macro Planning State):** Proposes dates, pacing, and daily themes without invoking external travel APIs, conserving tokens until macro consensus is reached.
- **Phase 2 (Micro Curation State):** Activated only after macro approval. Executes external tool calls, applies 2–3 km geo-clustering to prevent zigzag routes, and prepares alternative stops.
- **Model Abstraction Layer:** Built using LangChain/LangGraph model abstractions. DeepSeek-V4 serves as the primary reasoning engine, with transparent fallback to OpenAI (GPT-4o), Anthropic Claude, or Google Gemini to prevent vendor lock-in.
- **Typed State Persistence:** Checkpoints of `AgentState` are saved in PostgreSQL via `AsyncPostgresSaver` through an authenticated Data Access Layer (DAL).

3.2. AI Agent Data Pipeline

Figure 2 illustrates the unified AI agent state and data pipeline, connecting room context, two-phase LangGraph states, typed checkpoints, model abstraction, and resilient tooling:

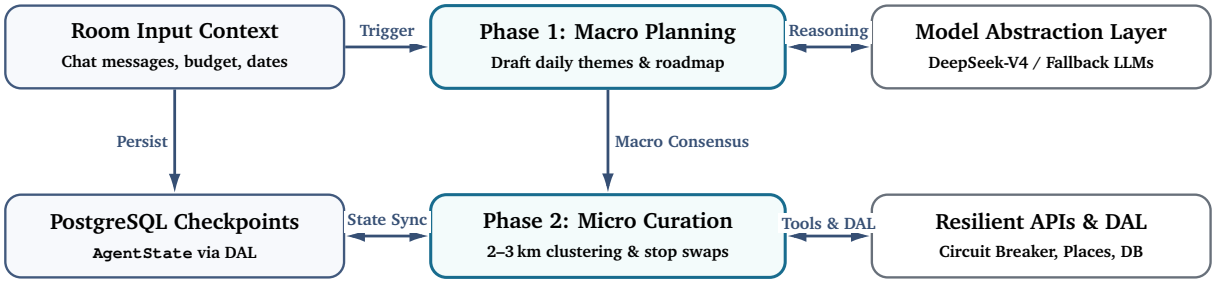

Figure 2: AI Agent state and data pipeline: two-phase LangGraph state execution with model abstraction and resilient tooling

3.3. Group Planning Workflow: Activity Diagram

Figure 3 illustrates the end-to-end user and AI interaction flow across two streamlined swimlanes:

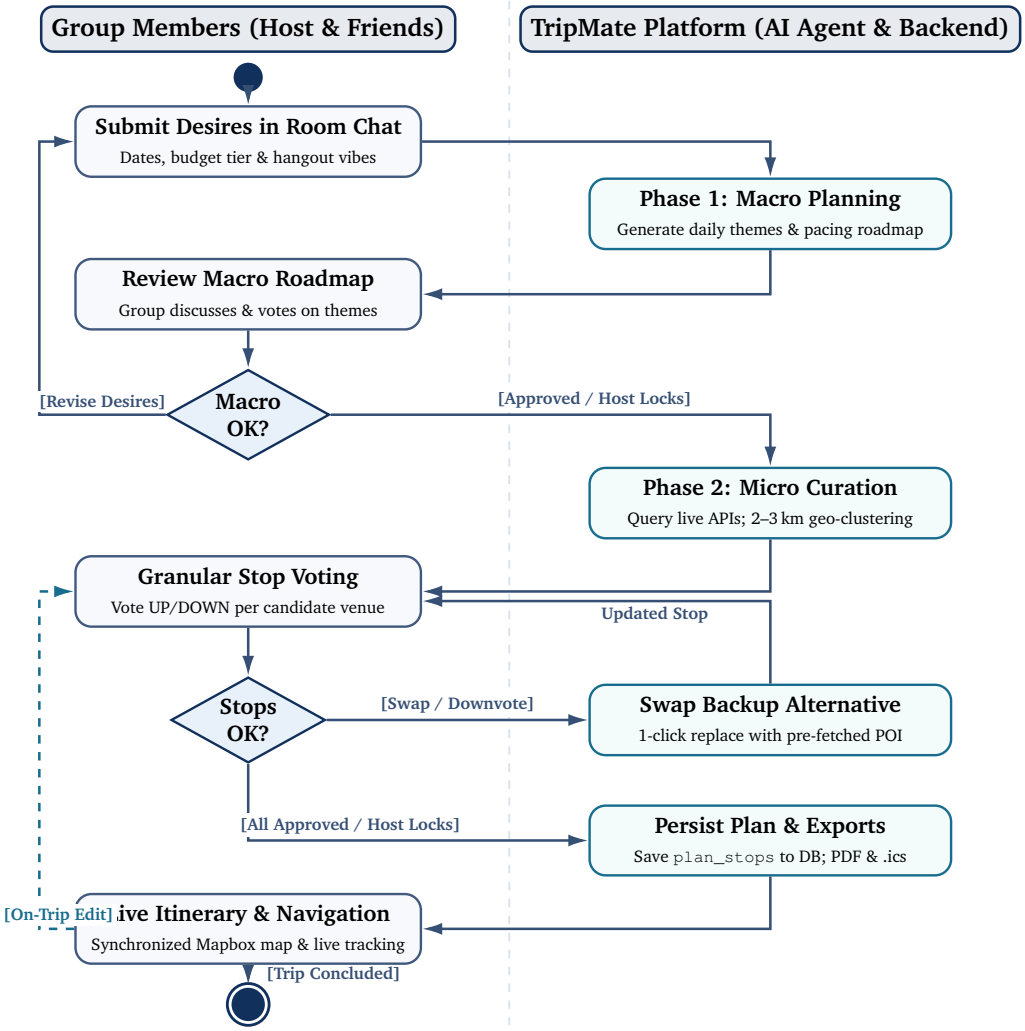

Figure 3: UML activity diagram: streamlined group planning workflow across user and AI system partitions

3.4. AI Reliability: Four-Tier Grounding Pipeline

To eliminate AI hallucinations (inventing fake places, closed venues, or unrealistic routes), TripMate AI verifies every recommendation through four simple steps (Figure 4):

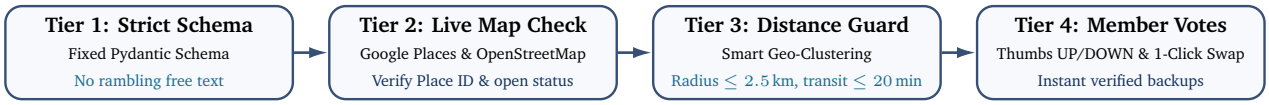

Figure 4: TripMate AI four-tier live grounding and hallucination prevention pipeline

- **Tier 1 (Strict Schema):** Restricts AI output to a fixed Pydantic format (name, category, duration, budget tier), preventing rambling or missing fields.
- **Tier 2 (Live Map Check):** Validates each venue via Google Places API and OpenStreetMap to ensure it actually exists, is open, and has correct hours.
- **Tier 3 (Distance Guard):** Uses Haversine distance and Mapbox to cluster half-day stops within 2.0–2.5 km ($\leq$ 20 min transit), avoiding zigzag travel.
- **Tier 4 (Voting & Backup Swap):** Members vote thumbs UP/DOWN on each stop; downvoted venues are instantly replaced with pre-verified backups in 1 click.

4. System Architecture, Technology Stack & Cloud Deployment

TripMate AI uses an asynchronous, decoupled architecture connecting the React SPA, FastAPI gateway, Supabase PostgreSQL storage, the unified LangGraph AI agent, and external travel APIs. Figure 5 presents the complete system architecture and highlights the primary communication interfaces (1)–(6) coordinating the platform across cloud hosting providers.

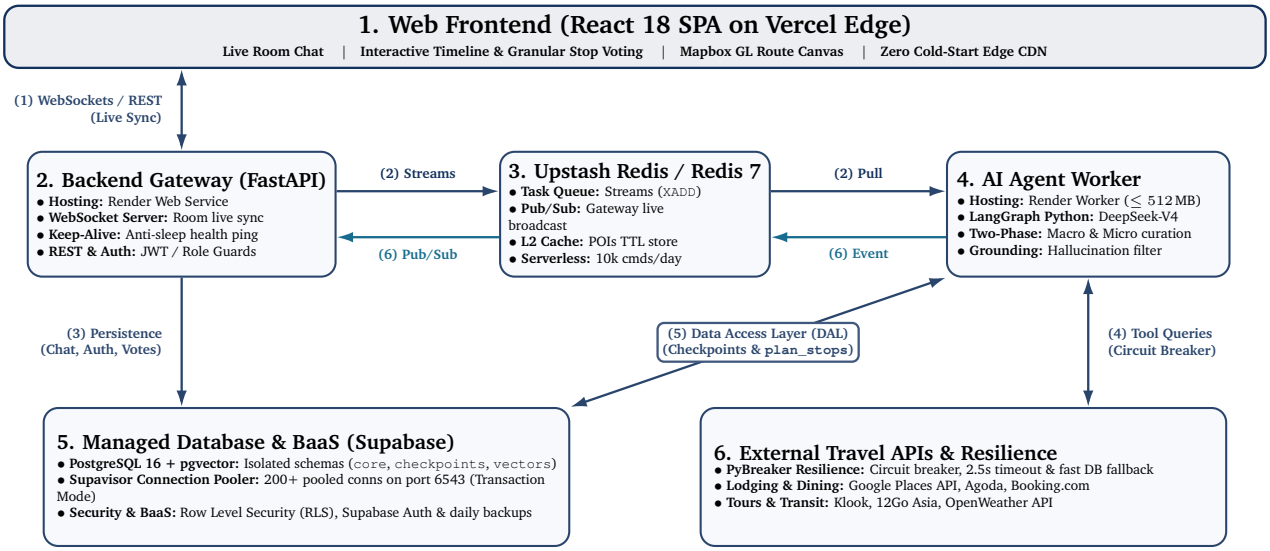

Figure 5: TripMate AI system architecture: component layers, cloud providers, and operational interfaces (1)–(6)

4.1. Technology Stack

Tier	Core Technology	Cloud Platform	Architectural Role
Frontend	React 18 + TypeScript	**Vercel**	Interactive timeline, collaborative voting cards, Mapbox GL canvas on Edge CDN.
Backend Gateway	FastAPI (Python 3.11+)	**Render.com**	REST endpoints, WebSocket room server, anti-sleep keep-alive pings.
AI Agent Layer	LangGraph Python	**Render.com**	Unified DeepSeek-V4 agent, two-phase states, memory bounded ≤ 280 MB.
	Model Abstraction	Cloud LLM APIs	Primary: DeepSeek-V4; Fallback: OpenAI GPT-4o, Claude, Gemini.
Database & Auth	PostgreSQL 16 + `pgvector`	**Supabase**	Relational persistence, Supavisor connection pooler (port 6543), RLS security.
Queue & Cache	Redis 7 / Redis Streams	**Upstash Redis**	Asynchronous task broker, WebSocket Pub/Sub broadcast, 24h TTL POI cache.
Resilience & Ops	PyBreaker / OpenTelemetry	Cloud Observability	External API circuit breakers (2.5s timeout), distributed tracing, LangSmith.

Table 1: Core technology stack and cloud platform distribution for TripMate AI

4.2. Cloud Deployment Architecture (Vercel, Supabase, Render)

TripMate AI is deployed across modern cloud platforms (Vercel, Supabase, Render) configured for high availability:

- **Frontend (Vercel):** React 18 SPA hosted on Vercel's Global Edge CDN with zero cold-start latency, automatic GitHub CI/CD, and custom domain SSL.
- **Backend Gateway & AI Worker (Render):** FastAPI ASGI application and LangGraph worker running in containerized environments (512 MB RAM). An automated keep-alive cron ping to `/api/healthz` prevents instance idling during active planning sessions.
- **Database & Auth (Supabase):** Managed PostgreSQL 16 with `pgvector` and Supabase Auth. The built-in Supavisor connection pooler (port 6543, Transaction Mode) multiplexes 200+ client sessions across 15 physical connections, maintaining high connection availability.
- **Task Queue & Cache (Upstash Redis):** Serverless Redis managing Redis Streams for asynchronous AI agent task dispatch and Redis Pub/Sub for live room WebSocket broadcasting.

4.3. End-to-End Operational Flow

Figure 6 visualizes the operational execution lifecycle of interfaces **(1)–(6)** from the System Architecture (Figure 5) as an end-to-end UML sequence diagram, showing the asynchronous decoupling of client interactions on Vercel, background AI processing on Render, resilient tool execution, and real-time room broadcasting. Table 2 summarizes each operational stage, execution mode, and dependent triggers.

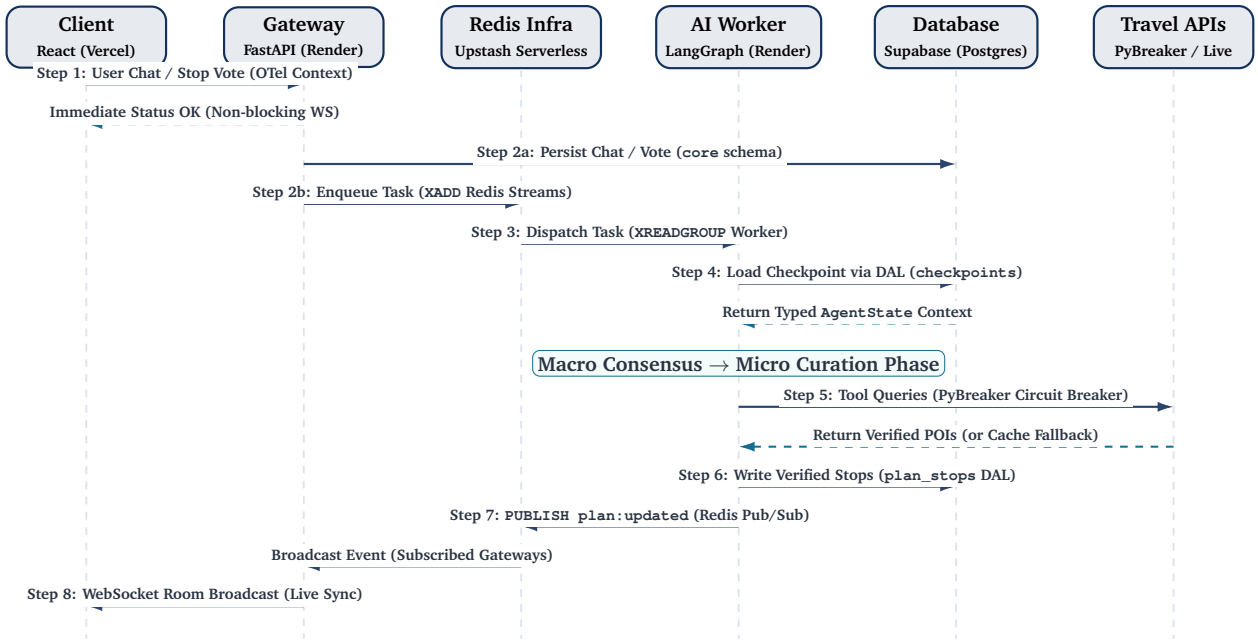

Figure 6: UML sequence diagram: end-to-end operational execution flow for the unified TripMate AI Agent

Step	Interface Path	Mode	Depends On	Operational Function
1	**(1)** Frontend → Gateway	Sync	User action	Transmits chat message or stop vote payload with OpenTelemetry context.
2a	**(3)** Gateway → Database	Sync	Step 1	Persists chat logs and vote counts into Supabase (`core` schema).
2b	**(2)** Gateway → Redis (Streams)	Async	Step 1	Dispatches non-blocking task to Redis Streams (`XADD`); returns immediate HTTP/WS ack.
3	**(2)** Redis (Streams) → AI Agent	Async	Step 2b	LangGraph worker consumes task via `XREADGROUP` consumer group.
4	**(5)** AI Agent ← Database	Async	Step 3	Loads active session checkpoint and context via DAL from `checkpoints` schema.
4b	LangGraph State Transition	Internal	Step 4	Evaluates consensus and transitions between Macro Planning and Micro Curation phases.
5	**(4)** AI Agent ↔ Travel APIs	Parallel	Step 4b	Queries live APIs via PyBreaker circuit breaker; falls back to cache/vector gems on timeout.
6	**(5)** AI Agent → Database	Async	Step 5	Writes normalized, verified venue records to `plan_stops` via DAL.
7	**(6)** AI Agent → Redis → Gateway	Async	Step 6	Publishes `plan:updated` via Redis Pub/Sub to synchronize all FastAPI gateway instances.
8	**(1)** Gateway → Frontend	Push	Step 7	Broadcasts refreshed itinerary to all connected room members via WebSockets.

Table 2: Unified AI Agent operational execution flow mapped to system interfaces

Dynamic On-the-Go Adaptability: When conditions change unexpectedly (e.g., sudden rain), a member chats: *"it is raining, find an indoor cafe nearby"*. FastAPI immediately returns an acknowledgment (non-blocking WebSocket), records the message in Supabase (`core` schema, Step 2a), and enqueues an asynchronous task to Redis Streams (Step 2b). The LangGraph AI Agent worker consumes the task (Step 3), restores the active session checkpoint from Supabase via DAL (Step 4), transitions directly to its Micro Curation state (Step 4b), queries nearby indoor venues in parallel through the resilient PyBreaker-guarded API and cache layer (Step 5), modifies **only the affected single row in `plan_stops`** in PostgreSQL (Step 6), publishes the update event to Redis Pub/Sub (Step 7), and FastAPI broadcasts the updated stop to all room members via WebSockets (Step 8).

5. System Targets, Evaluation & Engineering Foundations

5.1. Core System Targets: Latency, Capacity, Evaluation & Planet-Scale Scaling

TripMate AI defines four concrete, measurable performance targets:

- **Latency Targets:**
  - *User Actions:* < 80 ms WebSocket ACK for chat messages and votes.
  - *Initial AI Response (TTFT):* < 800 ms token streaming so users see answers immediately instead of waiting on a blank screen.
  - *Full Answer Times:* 6–12 s for macro roadmaps; 10–20 s for micro venue grounding; 3–6 s for 1-click swaps (Table 3).
- **Capacity Targets:**
  - *Concurrency:* 50–100 Concurrent Active Users (CCU) across 10–20 active rooms (3–6 users/room).
  - *Throughput:* 150–250 req/s on FastAPI gateway; 5–8 concurrent AI jobs with Redis queue buffer.
  - *Storage & Connections:* 30,000+ saved trips in PostgreSQL; 200+ user sessions multiplexed into 15 database connections via Supavisor pooler.
- **Evaluation Targets (Success Criteria):**
  - *AI Accuracy:* $\geq 98\%$ venues verified on Google Places (0% fake places); $\geq 95\%$ adherence to budget and distance constraints (≤ 2.5 km).
  - *Consensus Speed:* Reduce group decision time from 2–3 days down to < 15 minutes; $\geq 85\%$ initial stop approval.
  - *System Health:* $\geq 99.5\%$ uptime; < 100 ms multi-user WebSocket synchronization delay.
- **Planet-Scale Scalability Targets:**
  - Designed to scale globally without architectural rewrites: Vercel Global Edge CDN for sub-100 ms frontend delivery worldwide, stateless FastAPI and LangGraph workers on Render that scale horizontally, Redis Streams for queue buffering, and Supavisor connection pooling for database scalability.

Interaction Type	TTFT	Full Time	Operational Context
Member Action / Vote	< 80 ms	< 80 ms	Instant WebSocket ACK with atomic database update.
Quick Clarification / Q&A	< 600 ms	3–5 s	Direct LLM response for dietary, parking, or hours queries.
Macro Itinerary Drafting	< 800 ms	6–12 s	LangGraph Phase 1 themes and multi-day pacing roadmap.
Micro Spot Grounding	< 1.0 s	10–20 s	Parallel Google Places checks and route matrix calculation.
1-Click Swap / Revision	< 500 ms	3–6 s	Instant replacement with pre-verified backup venue.

Table 3: TripMate AI multi-turn interaction latency budget and response times

5.2. Evaluation Methodology: Verifying Agent & System Effectiveness

To empirically verify performance, TripMate AI evaluates both the AI Agent and the cloud System against concrete benchmarks:

5.2.1. AI Agent Evaluation (Planning Quality & Accuracy)

- **Grounding & Hallucination Test:** 100 automated test prompts across multiple cities check every venue against Google Places API. **Metric:** Verification Rate $=$ (Verified POIs with Place ID/Total POIs) $\geq 98\%$; 0% fake POIs.
- **Constraint Satisfaction Test:** Scripts verify that generated stops match budget limits, dietary preferences, and spatial clustering (≤ 2.5 km radius, ≤ 20 min transit). **Metric:** $\geq 95\%$ constraint pass rate.
- **User Acceptance & Speed Test:** Pilot trials with 10 real user groups (3–6 members) track voting telemetry and time to plan completion. **Metric:** $\geq 85\%$ first-recommendation approval; consensus reached in < 15 minutes.

5.2.2. System Performance Evaluation (Infrastructure & Reliability)

- **Latency & Tracing Test:** OpenTelemetry and LangSmith track end-to-end timing across Gateway $\rightarrow$ Queue $\rightarrow$ Agent $\rightarrow$ DB $\rightarrow$ WebSocket. **Metric:** TTFT < 800 ms, action ACK < 80 ms, room sync < 100 ms.
- **Load & Concurrency Stress Test:** Locust/k6 simulates 50–100 CCU in 10–20 rooms chatting, voting, and generating itineraries simultaneously. **Metric:** Sustained 150–250 req/s, error rate $< 0.5\%$, worker

memory ≤ 512 MB.
- **Fault Tolerance Test:** Chaos testing introduces 5s API delays and simulates primary LLM API outages. **Metric:** PyBreaker trips in 2.5s to return cached POIs; seamless fallback to secondary LLMs (GPT-4o/Gemini) without session drop.

5.3. Security, Multi-Tenant Privacy & Reliability

- **Data Privacy (Supabase RLS):** Database-level policies ensure users only access data in their own planning rooms.
- **Atomic Voting:** Atomic SQL updates and Redis counters prevent race conditions when multiple members vote simultaneously.
- **Input Sanitization:** Chat inputs are sanitized before LLM processing to prevent prompt injection attempts.
- **Session Recovery:** LangGraph execution checkpoints are saved in PostgreSQL, enabling instant recovery if a container restarts.

6. Expected Impact & Conclusion

TripMate AI represents a paradigm shift in collaborative group planning by unifying conversational AI intelligence, deterministic grounding, and modern web collaboration into a cohesive, scalable production system:

- **Sub-15-Minute Consensus:** Compresses days of scattered group chat debates into a structured, real-time 15-minute consensus session, eliminating organizer fatigue.
- **Zero-Hallucination Guarantee:** Grounded in physical reality via the Four-Tier Grounding Pipeline, strictly validating place IDs, operational status, opening hours, and geographic proximity against live travel APIs.
- **Production-Grade Cloud Performance:** Demonstrates exceptional engineering feasibility by orchestrating Vercel, Render, and Supabase while maintaining sub-800 ms initial response streaming and robust 50–100 CCU concurrency.
- **Enterprise-Grade Security & Resilience:** Enforces database-level isolation via Supabase RLS, atomic concurrency control for voting, and seamless checkpoint recovery for uncompromised data integrity.
- **Living, Adaptive Itineraries:** Couples transactional relational persistence with dynamic on-trip adaptation, allowing groups to modify single stops on the fly without recalculating untouched itinerary blocks.

TripMate AI turns group trip planning from an exhausting chore into an effortless, collaborative, and mathematically grounded experience.